

Milestones, tasks & timeframe

A normal project plan for two separate React Native apps and one Laravel backend, delivered by three frontend and three backend developers.

- 26-week controlled-release plan
- 13 two-week sprints
- Frontend + backend in parallel
- QA gates included

EXECUTIVE SUMMARY

What this timeframe means

6

Engineers from Month 1
3 React Native + 3 Laravel

26

Weeks for a controlled release, assuming early decisions and protected hardening

8–10

Months for the full visible scope at safer production quality

0

Dedicated QA currently identified — the largest delivery risk

Planning position: six months is achievable as a conditional, controlled-release target. It is not a safe unconditional commitment for every exported screen, full admin operations, complete edge cases, and production hardening.

WORKING ASSUMPTIONS

Confirmed versus still open

CONFIRMED

Technology and team

- Two separate React Native apps
- One Laravel backend
- Three frontend and three backend developers from Month 1
- PM owns requirements and UAT
- Infra team owns environments and operations

CONFIRMED

Commerce direction

- Third-party escrow
- Seller sends payment link in chat
- Seller manages logistics
- Seller submits tracking to buyer
- All visible Figma scope is desired

MUST DECIDE

Schedule blockers

- Escrow provider/API/webhooks
- Minimum admin scope and owner
- Compatibility data source and legal rights
- Refund/return/dispute rules
- Daily QA ownership and device matrix

MILESTONE OVERVIEW

Six-month delivery map

Each month ends with a working gate. A milestone is complete only when both apps, backend, automated tests, and acceptance checks agree on the same behavior.

PERIOD	PRIMARY MILESTONE	FRONTEND OUTCOME	BACKEND OUTCOME	EXIT GATE
Month 1 Weeks 1–4	Foundation and identity	Two app shells, shared packages, social login, profile and vehicle flows	API foundation, auth, roles, sellers, vehicle taxonomy, CI and documentation	Both apps authenticate against staging; user can save a default car
Month 2 Weeks 5–8	Core marketplace	User catalogue and seller product/listing CRUD	Products/SKUs, categories, media, stock, search and compatibility	Seller publishes an approved part; buyer finds it by selected vehicle
Month 3 Weeks 9–13	Conversation and first transaction slice	User/seller chat, product cards, order entry points	Realtime chat, order foundation, escrow adapter skeleton, notifications	Buyer and seller complete a staging chat-to-order journey
Month 4 Weeks 14–17	Escrow, fulfilment and supporting scope	Payment link, status, tracking, refund/return UI; discovery modules	Escrow webhooks, order state machine, tracking, refunds/returns, minimum admin	Paid staging order can ship, track, complete, cancel and refund consistently
Month 5 Weeks 18–21	Integration and operations	Cross-app regression, notifications, error states and beta builds	Admin/operations, moderation, reconciliation, performance and security	Feature-complete beta passes critical regression and operational review
Month 6 Weeks 22–26	Hardening, UAT and release	Store builds, fixes, accessibility, device coverage and release monitoring	Load/security, recovery, monitoring, production migration and rollback	UAT, finance/escrow checks and production-readiness sign-off pass

DETAILED MILESTONES

Frontend and backend work by month

Month 1
Weeks 1–4

Foundation, identity and vehicle profiles

Build reusable foundations first so two apps do not become two unrelated codebases.

START IMMEDIATELY

FRONTEND TASKS

- Create User and Seller app bundles in one shared repository
- Set up navigation, environments, API client, secure storage and feature flags
- Recreate design tokens and reusable components from PNG references
- Implement social login shell, profile and address forms
- Implement My Cars and vehicle selection in User App
- Set up unit/component tests, linting and beta build pipeline

BACKEND TASKS

- Initialize Laravel API, MySQL, queues, cache, object storage and CI
- Define API v1 responses, validation, errors, pagination and OpenAPI
- Implement social auth, users, tokens, devices, roles and permissions
- Create seller profile/type/application foundations
- Create vehicle brand, series, variant, year and user-car models
- Add logging, audit, monitoring, test factories and seed/import structure

VALIDATION & GATE

- Both apps log in against staging
- User can create/edit/default a car
- Seller onboarding draft saves correctly
- OpenAPI and mobile API client agree
- CI runs tests on every change

Dependencies: approved social providers, launch markets, initial vehicle dataset.

Month 2
Weeks 5–8

Product catalogue and compatibility

Deliver the spare-parts differentiator and seller listing workflow.

DATA SOURCE REQUIRED

FRONTEND TASKS

- User App: categories, product lists, details, media and seller profile
- User App: search, filters, sorting, favourites and compatible-car status
- Seller App: create/edit products, media, category, attributes, price and stock
- Seller App: vehicle compatibility selection and publication status
- Add loading, empty, validation, offline and error states
- Contract-test mobile models against API schema

BACKEND TASKS

- Products, seller SKUs/offers, categories, attributes and images
- Price, stock, publication state, ownership and moderation policies
- Vehicle compatibility mapping, source provenance and correction history
- Search, filters, sorting, pagination and selected-car matching
- Wishlist, reviews, seller replies and reports foundations
- Admin APIs for category, listing and compatibility review

VALIDATION & GATE

- Seller publishes an approved spare part
- Buyer sees it for the correct default car
- Unknown compatibility is not shown as compatible
- Stock and price validation pass
- Search response meets agreed performance

Dependencies: compatibility granularity, legal dataset/scrape source, approval policy.

Month 3
Weeks 9–13

Chat, orders and first end-to-end journey

Create the shared transaction spine before adding more supporting modules.

ESCROW API REQUIRED

FRONTEND TASKS

- Shared conversation list/thread, message cache and realtime connection
- Text, image, product and order cards with retry/read states
- User App: contact seller and create order entry point
- Seller App: buyer conversations and order context
- Initial buyer/seller order lists and status components
- Push-notification and deep-link handling

BACKEND TASKS

- Conversations, participants, messages, media and read state
- Realtime events, offline push, rate limits, block/report and retention
- Cart/order snapshots, order items and initial state machine
- Escrow provider interface and transaction/link model
- Notification templates/events and audit history
- Used-car and car-plate core APIs if capacity allows

VALIDATION & GATE

- Buyer opens product chat with correct context
- Messages deliver, retry and mark read
- Seller receives an order record
- Unauthorized users cannot read conversations
- First staging buyer-to-seller slice works

Dependencies: realtime provider/approach, message policy, escrow provider sandbox.

Month 4
Weeks 14–17

Escrow, tracking, refunds and supporting discovery

Complete the operational transaction loop and add remaining user-facing modules.

BUSINESS RULES REQUIRED

FRONTEND TASKS

- Seller sends allowlisted escrow payment-link card
- User sees link, expiry and provider-confirmed payment status
- Seller submits courier and tracking; buyer sees tracking card/history
- Cancellation, refund, return and exception screens in both apps
- Used cars, plates, workshop, EV charging and community screens
- Seller address/location and listing flows

BACKEND TASKS

- Create/record escrow links and verify signed idempotent webhooks
- Implement paid, expired, cancelled, disputed and refunded states
- Complete order state machine, stock rules and seller fulfilment
- Courier/tracking validation, event history and buyer notifications
- Cancellation, refund, return and dispute records
- Supporting listing/maps/community APIs and minimum operational admin

VALIDATION & GATE

- Duplicate webhook does not duplicate payment
- Arbitrary links are not shown as verified payments
- Paid order can ship, track and complete
- Cancel/refund/return states stay consistent across apps
- Admin can resolve basic exceptions

Dependencies: escrow webhook capability, refund/dispute policy, tracking requirements, admin owner.

Month 5
Weeks 18–21

Feature integration, operations and beta

Stop building isolated screens and prove complete journeys.

FEATURE-COMPLETE TARGET

FRONTEND TASKS

- Cross-app integration and shared status consistency
- Complete notifications, settings, reviews, reports and history
- Close empty/error/offline/permission/expired-link states
- Accessibility, localization and device compatibility review
- Performance tuning, crash reporting and beta distribution
- Fix PM beta feedback without expanding scope

BACKEND TASKS

- Complete admin APIs, approval queues and audit views
- Escrow exception reconciliation and operational exports
- Reported chat, user, seller, listing and review moderation
- Performance indexes, cache strategy and queue reliability
- Security review, abuse controls and data-retention jobs
- Staging migration, backup/restore and monitoring validation

VALIDATION & GATE

- Critical journeys pass automated regression
- Feature-complete beta distributed
- No unresolved critical authorization defects
- Escrow/tracking exception handling tested
- Operations can run MVP without database edits

Gate: feature freeze at end of Month 5 except approved launch blockers.

Month 6
Weeks 22–26

Regression, UAT and production readiness

Protect this month from new feature work.

LAUNCH GATE

FRONTEND TASKS

- Device/OS regression for both apps
- Social login, push, deep links, offline and upgrade tests
- Performance, accessibility and localization fixes
- App privacy declarations, signing and store submission
- Crash-free beta monitoring and launch-blocker fixes
- Versioning, forced/optional upgrade behavior and release notes

BACKEND TASKS

- Load, concurrency, webhook, permission and upload testing
- Escrow reconciliation and dispute/refund rehearsal
- Production migration, scheduled jobs and queue recovery
- Backup/restore, alert, incident and rollback rehearsal
- API documentation, runbooks and support handoff
- Only launch-blocking defects after code freeze

VALIDATION & GATE

- PM signs off actor-based UAT
- Infra signs off deploy/rollback/recovery
- Escrow provider staging reconciliation passes
- Critical and high launch blockers closed
- Management makes go/no-go decision

Rule: failed readiness gate moves the launch; it does not remove security or finance testing.

TWO-WEEK EXECUTION TIMETABLE

13 sprint plan

This is the working sequence for a six-month controlled release. Exact dates should be added after kickoff, holidays and leave are confirmed.

SPRINT	FRONTEND FOCUS	BACKEND FOCUS	VALIDATION / DELIVERABLE
S1 Weeks 1–2	Repo/app shells, tokens, navigation, API client, login UI	Laravel foundation, CI, API standard, auth schema, environments	Both app shells call staging health/auth endpoints
S2 Weeks 3–4	Social auth, profiles, addresses, My Cars, seller onboarding draft	OAuth/tokens, roles, profiles, seller base, vehicle/user-car APIs	Foundation gate: login + default car + seller draft
S3 Weeks 5–6	User catalogue/category/list/detail; seller product forms	Products, SKUs, categories, attributes, media, price and stock	Seller creates draft; user loads approved product
S4 Weeks 7–8	Search/filter/favourite; compatibility and publication UI	Compatibility pipeline, search/filter, approval, reviews/reports	Marketplace gate: selected-car matching works
S5 Weeks 9–10	Chat list/thread, message states, product cards	Realtime conversations/messages/media/read state/push	Buyer and seller exchange product-context messages
S6 Weeks 11–12	Order entry, buyer/seller order list and details	Order snapshots, items, initial states, notifications	Order visible consistently in both apps
S7 Week 13 + buffer	Escrow link-card prototype; used-car/plate start	Escrow adapter/link model; authorization and integration fixes	First complete staging journey; Month 3 gate
S8 Weeks 14–15	Payment status, seller ship/tracking, buyer tracking	Webhook verification, order states, courier/tracking events	Staging paid order ships and tracks
S9 Weeks 16–17	Cancel/refund/return; workshops/EV/community/supporting listings	Refund/return/dispute, maps/supporting APIs, admin minimum	Transaction gate including failure paths
S10 Weeks 18–19	Cross-app integration, notifications, missing states	Admin/operations, moderation, reconciliation, audit	Feature-complete candidate
S11 Weeks 20–21	Accessibility, localization, device fixes, beta builds	Performance, security, queues, retention, beta support	Feature freeze and beta gate
S12 Weeks 22–23	Full regression, store preparation, launch blockers	Load/security/concurrency, migrations, monitoring	UAT candidate; no new features
S13 Weeks 24–26	UAT fixes, store submission, release monitoring	Recovery/rollback, reconciliation, production release	Production-readiness sign-off and go/no-go

TEAM OWNERSHIP

Suggested allocation by developer

FRONTEND ROLE	PRIMARY OWNERSHIP	CONVE	BACKEND ROLE	PRIMARY OWNERSHIP	CONVE
FE Lead	Shared architecture, auth, API client, design system, releases, code review	Chat, integr	BE Lead	Foundation, identity, architecture, escrow/security, review	Commerce, p
FE 2	User App: vehicle, catalogue, discovery, buyer orders	Escrow statu	BE 2	Vehicle data, compatibility, catalogue, search, supporting modules	Admin, data
FE 3	Seller App: onboarding, listings, seller orders and fulfilment	Chat, payme	BE 3	Sellers, chat, orders, tracking, notifications, admin APIs	Escrow, integ

Ownership rule: every domain needs one primary engineer and one reviewer. The lead must not become the only person able to approve architecture, release, and finance-critical work.

QUALITY TIMETABLE

QA work across all six months

WHEN	DEVELOPER QA	PM / BUSINESS VALIDATION	INFRASTRUCTURE VALIDATION	REQUIRED EVIDENCE
Every change	Unit, component, feature, API contract and authorization tests	Acceptance criteria clarified before development	CI environment availability	Passing pipeline and review
Every sprint	Critical-path integration and regression	Sprint demo and acceptance feedback	Logs, job failures and environment issues reviewed	Demo, defect list, updated risks
Monthly gate	End-to-end journey, edge states, device smoke tests	Milestone acceptance against exit criteria	Deployment/rollback and monitoring check	Signed gate or re-baseline
Month 4	Webhook idempotency, escrow state, tracking, refund/return tests	Business rules and exception handling	Provider connectivity and secret rotation	Transaction test report
Month 5	Full cross-app regression, performance and security	Beta feedback triage without scope growth	Backup/restore and alert validation	Feature-freeze checklist
Month 6	Device/OS matrix, load, concurrency and launch blockers	Actor-based UAT and go/no-go recommendation	Production rehearsal, rollback, recovery and monitoring	Launch-readiness sign-off

Recommendation: add at least one dedicated QA engineer or equivalent test capacity by Month 3. PM-led UAT is essential, but it cannot replace daily regression across two apps, multiple devices, chat, escrow, and order states.

ESTIMATE AND CONTINGENCY

Which timeframe should management approve?

CONTROLLED RELEASE

6 months

Medium confidence only when scope locks, providers are ready, minimum admin is accepted and Month 6 is protected.

Likely deferral order

1. Advanced analytics
2. Community enhancements
3. Workshop/EV enhancements
4. Car-plate enhancements
5. Used-car enhancements

RECOMMENDED ESTIMATE

8–10 months

Most credible for all visible scope, complete admin operations, edge states, production quality and reasonable contingency.

Extra time supports

- Full supporting modules
- Robust disputes/refunds
- Device and localization polish
- Operational admin and reporting
- Safer rollout and defect recovery

HIGH-RISK ATTEMPT

6 months full

Low confidence without added QA/admin capacity, immediate P0 decisions and unusually low requirement/provider delay.

Failure signals

- Escrow undecided after Month 1
- Compatibility source not ready
- Admin starts in Month 5
- Feature work enters Month 6
- No daily regression owner

Full-scope extension path

EXTENSION	FOCUS	OUTCOME
Months 7–8	Deferred supporting modules, advanced admin, disputes/refunds, localization and device coverage	Full visible feature coverage and stronger operations
Months 9–10	Scale/performance, security remediation, pilot feedback, analytics, production stabilization	Safer full commercial launch with contingency

DECISION REGISTER

Questions that can change the schedule

P0	Launch definition and MVP boundary Beta, public MVP or full production? What may defer?	MANAGEMENT / PM
P0	Escrow provider and integration API-created links, signed webhooks, expiry, disputes, refunds, fees and reconciliation.	BUSINESS / FINANCE
P0	Compatibility data Legal source, matching depth, accuracy owner, updates and corrections.	PRODUCT / DATA
P0	Order and refund rules Payment confirmation, stock, cancellation, shipping deadline, dispute, return and refund.	PRODUCT / OPS
P1	Admin dashboard Minimum scope, UI owner, permissions, audit and exception handling.	OPERATIONS / PM
P1	QA and release ownership Daily functional QA, device matrix, app stores, regression and go/no-go approvers.	MANAGEMENT

P1

External providers

Social login, realtime chat, push, maps, storage, analytics and crash reporting.

**ENGINEERING /
INFRA**

MEETING OUTPUT

What to approve tomorrow

Required decisions

- Approve launch definition and protected MVP
- Approve six months as conditional or approve 8–10 months for full scope
- Select/shortlist escrow provider and validate capabilities
- Assign compatibility-data owner and legal source
- Approve minimum admin and QA ownership

Actions immediately after meeting

1. Record decisions and owners.
2. Convert relative weeks into calendar dates.
3. Assign developer domain ownership and reviewers.
4. Write acceptance criteria for Month 1 and Month 2 gates.
5. Start foundations while unresolved P0 items are actively closed.

D-Auto Club · Milestones, tasks and timeframe · Planning baseline for discussion, not an unconditional delivery commitment.